

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-108БВ-25

Студент: Басаргин М.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 04.10.26

Москва, 2026

Постановка задачи

Цель работы: приобретение практических навыков в управлении процессами в ОС и в обеспечении обмена данными между процессами посредством каналов.

Задание: составить и отладить программу на языке C++, которая работает с процессами и организует взаимодействие между ними. Основной процесс создаёт для решения задачи дочерний процесс, а взаимодействие между процессами осуществляется через каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в ходе работы. Пользоваться можно только системными вызовами.

Условие группы вариантов 1: родительский процесс создаёт дочерний. Первой строкой пользователь вводит в консоль родительского процесса имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передаёт команды пользователя через pipe1, который связан со стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передаёт данные в родительский процесс через pipe2. Результаты своей работы дочерний процесс пишет в созданный им файл.

Вариант 3.

Пользователь вводит команды вида «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс производит деление первого числа на последующие, а результат выводит в файл. Если происходит деление на 0, то дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип int. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void);` – создаёт дочерний процесс (копию текущего). В родителе возвращает PID ребёнка, в ребёнке – 0, при ошибке – -1.
- `int pipe(int *fd);` – создаёт канал (pipe). `fd[0]` – конец для чтения, `fd[1]` – конец для записи. Возвращает 0 при успехе и -1 при ошибке.
- `int dup2(int oldfd, int newfd);` – делает так, чтобы дескриптор `newfd` указывал туда же, куда `oldfd`. Использую, чтобы подключить каналы к стандартному вводу и выводу дочернего процесса.
- `int execl(const char *path, const char *arg, ...);` – заменяет текущий процесс другой программой (у меня это child). Список аргументов заканчивается `(char*)nullptr`.
- `ssize_t read(int fd, void *buf, size_t count);` – читает до count байт из дескриптора. Возвращает число прочитанных байт, 0 – если данных больше не будет, -1 – ошибка.
- `ssize_t write(int fd, const void *buf, size_t count);` – записывает байты в дескриптор. Может записать меньше, чем просили, поэтому вызываю её в цикле.
- `int open(const char *path, int flags, mode_t mode);` – открывает файл. Я использую флаги `O_WRONLY | O_CREAT | O_TRUNC`: только запись, создать файл, если его нет, и очистить, если он есть. Возвращает дескриптор или -1.
- `int close(int fd);` – закрывает дескриптор.
- `pid_t waitpid(pid_t pid, int *status, int options);` – родитель ждёт завершения дочернего процесса.

- `ssize_t readlink(const char *path, char *buf, size_t bufsiz);` – читает, на что указывает символическая ссылка. Использую для `/proc/self/exe`, чтобы узнать папку, где лежит `parent`, и запустить `child` из той же папки.
- `sighandler_t signal(int signum, sighandler_t handler);` – задаёт обработку сигнала. Использую `signal(SIGPIPE, SIG_IGN)`, чтобы запись в закрытый канал не убивала программу.
- `void _exit(int status);` – сразу завершает процесс. Использую в дочернем процессе, если после `fork` что-то пошло не так (например, не запустился `exesl`).

Моя программа состоит из двух отдельных файлов: `parent.cpp` (родительский процесс) и `child.cpp` (дочерний процесс). Они компилируются отдельно, получаются два исполняемых файла: `parent` и `child`.

Как работает `parent`

1. Выводит приглашение и читает с клавиатуры имя файла (системный вызов `read` из дескриптора 0).
2. Создаёт два канала: `pipe1` (от родителя к ребёнку, по нему идут строки с числами) и `pipe2` (от ребёнка к родителю, по нему ребёнок отвечает).
3. Делает `fork`. В дочернем процессе закрывает ненужные концы каналов, через `dup2` подключает `pipe1` к стандартному вводу, а `pipe2` к стандартному выводу, и запускает программу `child` через `exesl`, передавая ей имя файла. Путь к `child` родитель находит через `readlink("/proc/self/exe")`, поэтому `child` должен лежать в одной папке с `parent`.
4. В родительском процессе закрывает ненужные концы каналов и входит в цикл: читает строку с числами, отправляет её в `pipe1` и ждёт один байт ответа из `pipe2`.
5. По ответу решает, что делать: «k» – всё нормально, читаем следующую строку; «e» – ввод некорректный, выводит сообщение и продолжает; «z» – было деление на 0, выводит сообщение и завершает работу.
6. Пустая строка или `Ctrl+D` – выход из цикла. Перед завершением закрывает каналы и вызывает `waitpid`, чтобы дождаться дочерний процесс.

Как работает `child`

1. Получает имя файла в `argv[1]` и открывает его через `open` (файл создаётся, если его нет, и очищается, если он есть).
2. Читает из стандартного ввода (на самом деле это `pipe1`) и складывает прочитанное в строку, пока не появится символ перевода строки – так получается одна целая строка с числами.
3. Разбирает строку на числа типа `int` (функция `parse`). Если в строке есть что-то кроме чисел или число не помещается в `int`, отправляет родителю «e».
4. Делит первое число на второе, результат на третье и так далее (целочисленное деление, считаю в `long long`, чтобы не было переполнения). Проверка на 0 происходит именно здесь, в `child`, перед каждым делением.

5. Если встретился 0, отправляет родителю «z», закрывает файл и завершается. Иначе записывает результат в файл (одно число на строку) и отправляет «k».

6. Когда parent закрывает pipe1, read возвращает 0 – данных больше не будет, и child заканчивает работу.

Все системные вызовы, которые могут завершиться ошибкой (pipe, fork, dup2, exec1, open, write), проверяются. При ошибке программа выводит сообщение в стандартный поток ошибок и завершается. Ненужные концы каналов в каждом процессе закрываются: как я понял, если этого не сделать, child никогда не получит признак конца данных.

Код программы

child.cpp

```
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <string>
#include <vector>

static bool write_all(int fd, const std::string& s) {
    const char* p = s.c_str();
    size_t left = s.size();
    while (left > 0) {
        ssize_t n = write(fd, p, left);
        if (n < 0) {
            if (errno == EINTR) continue;
            return false;
        }
        p += n;
        left -= n;
    }
    return true;
}

// разбор строки в числа типа int; false при некорректном вводе
static bool parse(const std::string& s, std::vector<int>& out) {
    out.clear();
    size_t i = 0, n = s.size();
    while (true) {
        while (i < n && (s[i] == ' ' || s[i] == '\t' || s[i] == '\r')) i++;
        if (i >= n) break;

        bool neg = false;
        if (s[i] == '-' || s[i] == '+') {
            neg = (s[i] == '-');
            i++;
        }
    }
}
```

```

    if (i >= n || s[i] < '0' || s[i] > '9') return false;

    long long v = 0;
    while (i < n && s[i] >= '0' && s[i] <= '9') {
        v = v * 10 + (s[i] - '0');
        if (v > 2147483648LL) return false; // выход за int
        i++;
    }
    if (i < n && s[i] != ' ' && s[i] != '\t' && s[i] != '\r') return false;
    if (neg) v = -v;
    if (v > 2147483647LL || v < -2147483648LL) return false;
    out.push_back((int)v);
}
return !out.empty();
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        write_all(STDERR_FILENO, "child: не передано имя файла\n");
        return 1;
    }

    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        write_all(STDERR_FILENO, "child: не удалось открыть файл\n");
        return 1;
    }

    std::string pending;
    char buf[256];
    std::vector<int> nums;

    while (true) {
        ssize_t n = read(STDIN_FILENO, buf, sizeof(buf));
        if (n < 0) {
            if (errno == EINTR) continue;
            write_all(STDERR_FILENO, "child: ошибка чтения\n");
            close(fd);
            return 1;
        }
        if (n == 0) break; // родитель закрыл pipe1
        pending.append(buf, n);

        size_t pos;
        while ((pos = pending.find('\n')) != std::string::npos) {
            std::string line = pending.substr(0, pos);
            pending.erase(0, pos + 1);

            if (!parse(line, nums)) {

```

```

        write_all(STDOUT_FILENO, "e");
        continue;
    }

    // делим первое число на последующие
    long long result = nums[0];
    bool zero = false;
    for (size_t i = 1; i < nums.size(); i++) {
        if (nums[i] == 0) { zero = true; break; }
        result /= nums[i];
    }

    if (zero) {
        write_all(STDOUT_FILENO, "z"); // сообщаем родителю
        close(fd);
        return 0;
    }

    if (!write_all(fd, std::to_string(result) + "\n")) {
        write_all(STDERR_FILENO, "child: ошибка записи в файл\n");
        close(fd);
        return 1;
    }
    write_all(STDOUT_FILENO, "k");
}

close(fd);
return 0;
}

```

Протокол работы программы

Тест 1. Обычная работа

```
$ ./parent
```

Введите имя файла: out.txt

Вводите числа через пробел (пустая строка или Ctrl+D для выхода):

```
100 5 2
```

```
-81 3 3
```

```
7 10 3
```

```
1000 10 10 10
```

```
$ cat out.txt
```

```
10
```

```
-9
```

```
0
```

```
1
```

Проверка вручную: $100 / 5 / 2 = 10$; $-81 / 3 / 3 = -9$; $7 / 10 = 0$ (деление целочисленное), $0 / 3 = 0$; $1000 / 10 / 10 / 10 = 1$.

Тест 2. Деление на 0

```
$ ./parent
```

Введите имя файла: out2.txt

Вводите числа через пробел (пустая строка или Ctrl+D для выхода):

```
50 5
```

```
10 0 5
```

Деление на 0. Завершение работы.

```
$ cat out2.txt
```

```
10
```

Строка «10 0 5» в файл ничего не записала, оба процесса завершились сами.

Тест 3. Некорректный ввод

```
$ ./parent
```

Введите имя файла: out3.txt

Вводите числа через пробел (пустая строка или Ctrl+D для выхода):

```
abc
```

Некорректный ввод, ожидается: число число ...

```
12 x
```

Некорректный ввод, ожидается: число число ...

```
20 2
```

```
-20 -2
```

```
9999999999 1
```

Некорректный ввод, ожидается: число число ...

```
$ cat out3.txt
```

```
10
```

Некорректные строки (буквы и число, не помещающееся в int) пропускаются, программа продолжает работать.

Тест 4. Пустое имя файла

```
$ ./parent
```

Введите имя файла:

Ошибка: имя файла не задано

Вывод

В этой лабораторной работе было реализовано две программы – parent и child, которые обмениваются данными через два канала (pipe). Родитель принимает от пользователя строки с числами, а ребёнок делит числа, пишет результат в файл и сообщает родителю результат одной буквой. Так я на практике познакомился с системными вызовами fork, pipe, dup2, execl, read, write и waitpid и лучше понял, как работают процессы и каналы.